

Algoritmos e Lógica de Programação

Aula 02 — Operadores Relacionais, Lógicos e Estruturas de Decisão

2º Semestre de 2025

Prof. Marcelo Amorim



Recapitulando: operadores aritméticos

- $+$ $-$ $*$ $/$: as quatro operações básicas (soma, subtração, multiplicação e divisão).
- $\%$: o resto da divisão — muito usado para descobrir se um número é par/ímpar, extrair dígitos e converter unidades.
- Cuidado: a divisão entre dois int trunca o resultado (a parte decimal é descartada).

Hoje vamos aprender operadores que respondem verdadeiro ou falso — e as estruturas que decidem o que o programa faz com essa resposta.



Operadores relacionais

Comparam dois valores. O resultado de uma comparação é sempre um bool: true (verdadeiro) ou false (falso).

$>$ Maior que	$<$ Menor que	$\geq$ Maior ou igual	$\leq$ Menor ou igual	$==$ Igual a	$\neq$ Diferente de
$5 > 3$ → true	$3 < 5$ → true	$5 \geq 5$ → true	$4 \leq 5$ → true	$5 == 5$ → true	$5 \neq 3$ → true

Atenção: `==` compara valores; `=` sozinho é usado para atribuir um valor a uma variável.

Operadores lógicos

Combinam expressões que já resultam em bool, formando condições mais complexas.

&& E (AND)

as duas precisam ser verdadeiras

|| OU (OR)

basta uma ser verdadeira

! NÃO (NOT)

inverte o valor: true vira false

Tabela-verdade

A	B	A && B	A B
V	V	V	V
V	F	F	V
F	V	F	V
F	F	F	F

O ! (NÃO) inverte qualquer valor: $!V = F$ e $!F = V$.



Diferença entre E (&&) e OU (||)

E (&&) — as duas precisam ser verdadeiras

“Para dirigir, eu preciso ter mais de 18 anos E ter carteira de motorista.” Se faltar qualquer uma das duas, não posso dirigir.

```
idade >= 18 && temCarteira
```

OU (||) — basta uma ser verdadeira

“Posso pagar em dinheiro OU no cartão.” Qualquer uma das duas formas já resolve o pagamento.

```
pagaDinheiro || pagaCartao
```



Expressões lógicas (rodada 1 de 3)

1. `(5 > 3) && (3 >= 7)`

2. `(10 % 2 == 0) || (7 < 3)`

3. `!(4 > 10)`



Resposta — rodada 1

```
1. (5 > 3) && (3 >= 7)  
→ true && false → falso
```

```
2. (10 % 2 == 0) || (7 < 3)  
→ true || false → verdadeiro
```

```
3. !(4 > 10)  
→ !(false) → verdadeiro
```



Expressões lógicas (rodada 2 de 3)

1. `(5 > 3) && ((3 >= 7) || (5 + 3 >= 8))`

2. `(2 == 2) && (4 != 4)`

3. `(8 % 3 == 2) || (1 > 0 && 0 > 1)`



Resposta — rodada 2

1. `(5 > 3) && ((3 >= 7) || (5 + 3 >= 8))`

→ `true && (false || true)` → verdadeiro

2. `(2 == 2) && (4 != 4)`

→ `true && false` → falso

3. `(8 % 3 == 2) || (1 > 0 && 0 > 1)`

→ `true || (true && false)` → verdadeiro



Expressões lógicas com variáveis (rodada 3 de 3)

```
int x = 4, y = 10;  
bool ehPar = true;  
bool ehGrande = false;
```

1. `ehPar && (x + y) % 2 == 0`

2. `(x > y) || (!ehGrande && x < 10)`

3. `(y % x != 2) && (ehPar || ehGrande)`

Resposta — rodada 3

```
int x = 4, y = 10;  
bool ehPar = true;  
bool ehGrande = false;
```

```
1. ehPar && (x + y) % 2 == 0  
→ true && (14 % 2 == 0) → true && true → verdadeiro
```

```
2. (x > y) || (!ehGrande && x < 10)  
→ false || (true && true) → verdadeiro
```

```
3. (y % x != 2) && (ehPar || ehGrande)  
→ (10 % 4 != 2) → false → falso
```



Estrutura de decisão: if / else

O if executa um bloco de código somente quando uma condição é verdadeira. Se a condição for falsa, esse bloco é simplesmente pulado.

```
if (idade >= 18) {  
    Console.WriteLine("Maior de idade");  
}
```

O if nem sempre precisa de else!

O else só entra quando existe uma ação alternativa para quando a condição é falsa. Se não há nada a fazer nesse caso, o if sozinho já resolve.



Sintaxe do if / else: os principais casos (1/2)

Caso 1 — comando único, sem chaves

```
if (condição)
    comando1();
```

Caso 2 — bloco de comandos, sem else

```
if (condição) {
    comando1();
    comando2();
    // ...
}
```

Caso 3 — comando único, com else

```
if (condição)
    comando1();
else
    comando2();
```

Sem chaves {}, apenas o próximo comando pertence ao if (ou ao else). Com chaves, todo o bloco pertence a ele.



Sintaxe do if / else: os principais casos (2/2)

Caso 4 — bloco no if, else de 1 linha

```
if (condição) {  
    comando1();  
    comando2();  
    // ...  
} else  
    comando3();
```

Caso 5 — if de 1 linha, bloco no else

```
if (condição)  
    comando1();  
else {  
    comando2();  
    comando3();  
    // ...  
}
```

Caso 6 — bloco no if e no else

```
if (condição) {  
    comando1();  
    comando2();  
    // ...  
} else {  
    comando3();  
    comando4();  
    // ...  
}
```

As chaves definem onde o bloco começa e termina; sem elas, apenas uma linha pertence à estrutura.

Exercício guiado: par ou ímpar

Vamos programar juntos: leia um número inteiro e informe se ele é par ou ímpar.

1. Ler um número inteiro.
2. Verificar se o resto da divisão desse número por 2 é igual a 0.
3. Exibir “Par” ou “Ímpar”, conforme o resultado.

Dica: use o operador % que já vimos, dentro de um if / else.

Decisão encadeada: else if

Quando existem mais de duas possibilidades, encadeamos vários else if, um depois do outro.

```
if (nota >= 9)
    conceito = "A";
else if (nota >= 7)
    conceito = "B";
else if (nota >= 5)
    conceito = "C";
else
    conceito = "D";
```

As condições são testadas em ordem, de cima para baixo; assim que uma é verdadeira, as demais nem são checadas.

Decisão aninhada: if dentro de if

Às vezes só faz sentido verificar uma segunda condição depois que a primeira já foi verdadeira. Colocamos um if dentro de outro if.

```
if (usuarioValido) {  
    if (senhaValida) {  
        Console.WriteLine("Acesso permitido");  
    } else {  
        Console.WriteLine("Senha incorreta");  
    }  
} else {  
    Console.WriteLine("Usuário não encontrado");  
}
```

Repare que só chegamos a checar a senha se o usuário já for válido — é exatamente esse o papel do aninhamento.



Estrutura de múltipla escolha: switch

Quando comparamos uma única variável a vários valores possíveis, o switch deixa o código mais organizado que uma cadeia de else if.

```
switch (diaSemana)
{
    case 1: Console.WriteLine("Domingo"); break;
    case 2: Console.WriteLine("Segunda"); break;
    case 3: Console.WriteLine("Terça"); break;
    // ...
    default: Console.WriteLine("Inválido"); break;
}
```

O break encerra o case; o default cobre os valores que não bateram com nenhum case. Para intervalos ou condições compostas, o else if continua sendo a melhor opção.



Exercício 1

Reajuste salarial

Dado o valor atual do salário de um funcionário, informe o valor do salário reajustado, de acordo com os percentuais a seguir.

1. Ler o salário atual do funcionário.
2. Aplicar o percentual de reajuste conforme a faixa em que o salário se encaixa.
3. Exibir o novo salário, já reajustado.

Regras

até R\$ 2.000,00 → +50%

R\$ 2.000,01 a R\$ 4.999,99 → +20%

demais ($\geq$ R\$ 5.000,00) → +10%



Exercício 2

IMC com classificação

O IMC de uma pessoa é dado pelo peso (kg) dividido pelo quadrado da altura (m). Dados peso e altura, informe a situação da pessoa.

1. Ler o peso, em quilogramas.
2. Ler a altura, em metros.
3. Calcular o IMC: $\text{peso} / (\text{altura} * \text{altura})$.
4. Classificar e exibir a situação, conforme as faixas ao lado.

Regras

$\text{imc} \leq 18.5 \rightarrow \text{magro}$

$18.5 < \text{imc} \leq 25.0 \rightarrow \text{normal}$

$25.0 < \text{imc} \leq 30.0 \rightarrow \text{sobrepeso}$

$\text{imc} > 30.0 \rightarrow \text{obeso}$



Exercício 3

Ano bissexto

Crie um programa que solicite um ano e verifique se ele é bissexto, usando a condição ao lado.

1. Ler o ano.
2. Verificar a condição de ano bissexto.
3. Exibir se o ano é ou não é bissexto.

Condição

```
(ano % 4 == 0 &&  
ano % 100 != 0)  
|| (ano % 400 == 0)
```



Exercício 4

Maior de 3 números

Ler 3 números inteiros e apresentar o maior deles.

1. Ler os três números inteiros.
2. Comparar os valores entre si.
3. Exibir o maior deles.

Dica: pense em usar if/else if encadeados, ou if aninhados, comparando dois números de cada vez.



Exercício 5

Ordenar 3 números

Ler 3 números inteiros e apresentá-los em ordem crescente.

1. Ler os três números inteiros.
2. Comparar os valores para descobrir a ordem entre eles.
3. Exibir os três números em ordem crescente.

Dica: são várias comparações possíveis — vale desenhar num papel todos os casos de ordem antes de programar.



Exercício 6

Confirmação flexível

Ler a resposta do usuário para uma pergunta do tipo sim/não e interpretar como afirmativa ou negativa, aceitando variações como Sim, sim, SIM, S, s (e equivalentes para não).

1. Ler o texto digitado pelo usuário.
2. Padronizar o texto (maiúsculas/minúsculas e espaços) antes de comparar.
3. Verificar se a resposta corresponde a uma afirmação ou a uma negação.
4. Exibir o resultado interpretado.

Ferramentas novas para esse exercício:

- `.ToUpper()` / `.ToLower()` — converte a string inteira para maiúsculas ou minúsculas
- `.Trim()` — remove espaços em branco no início e no fim da string
- `==` — em C#, já compara o valor das strings — diferente de Java, que exige `.Equals()` para isso



Exercício 7

Usuário e domínio de um e-mail

Ler um e-mail digitado pelo usuário e exibir separadamente a parte antes do @ (usuário) e a parte depois do @ (domínio).

1. Ler o e-mail digitado pelo usuário.
2. Encontrar a posição do caractere @ dentro do texto.
3. Extrair a parte antes e a parte depois dessa posição.
4. Exibir as duas partes separadamente.

Ferramentas novas para esse exercício:

- `.IndexOf('@')` — retorna a posição (índice) em que o caractere aparece na string
- `.Substring(início, tamanho)` — extrai um trecho da string a partir de uma posição



Recapitulando

Hoje vimos:

- Operadores relacionais (> < >= <= == !=), que sempre retornam um bool.
- Operadores lógicos (&&, ||, !) e a tabela-verdade de cada um.
- Estrutura de decisão if/else, e que o else é opcional.
- Todos os casos de sintaxe do if/else, com linha única e com blocos.
- Decisão encadeada (else if), decisão aninhada, e a estrutura switch.
- Prática com 7 exercícios aplicando tudo isso, incluindo métodos de string como ToUpper, Trim, IndexOf e Substring.

Próxima aula: Estrutura de Repetição